University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: Robotics Simulations Using PyBullet and ROS2

Programme title: MSc in Artificial Intelligence

Name: Sanket Sadashiv Gadge

Year: 2025

CONTENTS

CONTENTS 2
Task 1: PyBullet World Building with URDFs 5
INTRODUCTION 5
CHAPTER ONE - PROJECT DEFINITION 6
1.1 Custom Model Creation 6
1.2 Scene Composition: 6
1.3 Multi-View Rendering 6
CHAPTER TWO - DESIGN AND IMPLEMENTATION 7
2.1 Making URDF Files 7
2.1.1 Platform URDF: 7
2.2 Scene Creation 9
2.3 Using Virtual Camera 9
2.4 Two wheel robot 11
CHAPTER THREE - EXECUTION AND TESTING 12
3. 1 Screenshots: 13
3.2 Testing 14
CHAPTER FOUR - EVALUATION AND REFLECTION 15
4.1 Challenges Faced and Solutions 15
4.1.1 Challenge 1: URDF Complexity 15
4.1.2 Challenge 2: Camera Angles 15
4.1.3 Challenge 3: Overwriting pybullet objects 15
4.2 Potential Improvements 16
4.2.1 More Dynamic Obstacles: 16
CONCLUDING REMARKS 17
Task 2: Autonomous Robot Simulation with ROS2 (TheConstruct.ai) 18
CHAPTER ONE - PROJECT DEFINITION 18
1.1 Project Structure 18
1.2 Tools and Libraries Used 18
CHAPTER TWO - DESIGN AND IMPLEMENTATION 19
2.1 Environment and robot/object descriptions 19
2.2 System Components 20
2.3 Sensor Integration: 20
2.3.1 LIDAR Integration 20
2.3.2 Camera Integration 20
2.3.3 Human-Robot Interaction (HRI) 21
2.3.4 HRI Features: 21
2.4 Key algorithms & how applied 21
2.4.1 LIDAR obstacle detection 21

2.4.2 Camera-based goal detection (color segmentation)..... 21
2.4.3 Finite State Machine (FSM)..... 22
CHAPTER THREE - EXECUTION AND TESTING.....23
3.1 How to run (TheConstruct.ai RosJect).....23
3.2 Test cases & outcomes.....23
3.3 Screenshots.....24
CHAPTER FOUR - EVALUATION AND REFLECTION..... 27
4.1 Challenges faced and solutions.....27
4.1.1 URDF wall orientation & texture.....27
4.1.2 ROS2 packaging & launch.....27
4.1.3 Wrong Gazebo launch file.....27
4.1.4 LIDAR rays clutter.....27
4.1.5 Camera/goal misclassification..... 27
4.2 Potential improvements..... 27
4.3 Reflection: mapping to real-world robotics.....28
CONCLUDING REMARKS..... 29
BIBLIOGRAPHY..... 30
APPENDIX (if necessary)..... 31

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

SANKET GADGE

.....

Date: 06/10/2025

Task 1: PyBullet World Building with URDFs

INTRODUCTION

Simulation in robotics is very important now. It helps for making quick models, testing codes, and checking the system. We don't have to spend much money or worry about breaking real robots. PyBullet is a python tool for Bullet physics and it is a simple and easy platform for such simulations (Coumans and Bai). This task is about using PyBullet for making a robot scene that is static and captured with three different camera angles. The goal of this simulation is to learn robotics in a 2d environment by creating our own worlds and training our robot to sustain in that virtual world. This is important because as per traditional methods, creating a robot and testing it in real-world environments can be costly and difficult , to avoid that we can use PyBullet to train and test our robot.

CHAPTER ONE - PROJECT DEFINITION

Here I had to make and show a fixed virtual scene inside the PyBullet physics simulator.

The main goals for this task are these :

1.1 Custom Model Creation

Creating my own URDF files for a minimum of four different static objects (for example table, pillar, wall) and also one robot with two wheels.

1.2 Scene Composition:

Creating a virtual scene by putting at least seven static objects and the robot in some proper arrangement.

1.3 Multi-View Rendering

Setting up and using the PyBullet camera to take three different pictures of the scene (either from top, from side, etc.). The static objects must not move, but teleport the robot be in different place in all three pictures, to look like it is moving. All the objects must be seen a little bit in every picture.

CHAPTER TWO - DESIGN AND IMPLEMENTATION

2.1 Making URDF Files

I had an idea to create a scene like "Takeshi's Castle". So, I made four different platforms with obstacles on them, and also a two-wheeled robot.

The main parts of a URDF file are:

- `<robot>`: The main tag.
- `<link>`: A body part of the robot.
- `<visual>`: How the link looks (shape, color).
- `<collision>`: The shape used for physics calculation.
- `<inertial>`: Mass and inertia properties.

Here are some examples of my URDF file:

2.1.1 Platform URDF:

```
<robot name="robot">

  <link name="base_footprint"></link>

  <joint name="base_joint" type="fixed">

    <parent link="base_footprint" />

    <child link="base_link" />

    <origin xyz="0 0 1" rpy="0 0 0" />

  </joint>

  <link name="base_link">

    <visual>

      <origin xyz="0 0 0" rpy="0 0 0" />

      <geometry>

        <box size="2 2 2" />
```

```
      </geometry>
      <material name="base_link-material">
        <color rgba="0.03560131486909763 0.768151147242580
0.0528606470161647 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <box size="1 1 1" />
      </geometry>
    </collision>
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="1" />
      <inertia ixx="0.16666666666666666" ixy="0" ixz="0"
iyy="0.16666666666666666" iyz="0" izz="0.16666666666666666" />
    </inertial>
  </link>
</robot>
```

2.2 Scene Creation

Here's the code for my the class Scene, which loads the urdfs and creates a fixed scene. I have used useFixedBase=True flag to set the loaded objects to fixed location.

```python
class Scene:
    def __init__(self, start_x=0, start_y=0):
        self.objects = []
        self.start_x, self.start_y = start_x, start_y

    def build_platforms(self):
        """Create ground + surrounding walls + some obstacles"""
        p.setAdditionalSearchPath(pybullet_data.getDataPath())
        plane = p.loadURDF("plane.urdf")
        texture_id = p.loadTexture("brick.png")
        self.objects.append(plane)

        platform_1 = p.loadURDF("platform_1.urdf", [self.start_x, self.start_y, 0.0], p.getQuaternionFromEuler([0,0,1.57]), useFixedBase=True)
        p.changeVisualShape(platform_1, -1, textureUniqueId=texture_id)
        platform_2 = p.loadURDF("platform_2.urdf", [self.start_x + 2, self.start_y, 0.0], p.getQuaternionFromEuler([0,0,1.57]), useFixedBase=True)
        p.changeVisualShape(platform_2, -1, textureUniqueId=texture_id)

        platform_3 = p.loadURDF("platform_3.urdf", [self.start_x + 2, self.start_y + 2, 0.0], p.getQuaternionFromEuler([0,0,1.57]), useFixedBase=True)
        platform_4 = p.loadURDF("platform_4.urdf", [self.start_x , self.start_y+2, 0.0], p.getQuaternionFromEuler([0,0,1.57]), useFixedBase=True)

        for plfm in [platform_1, platform_2, platform_3, platform_4]:
            self.objects.append(plfm)
        return self.objects
```

2.3 Using Virtual Camera

I have created a camera for capturing the images from different angles.

I use the following set_view function to set the camera at a certain angle calculated using yaw, pitch and roll. It uses pybullet's computeViewMatrixFromYawPitchRoll() function to get the view matrix.

```python
def set_view(self, view="top", yaw=None, pitch=None, roll=0):
    """Set camera view by predefined name or custom yaw/pitch"""
    if view == "top":
        yaw, pitch = 0, -89
    elif view == "left":
        yaw, pitch = 90, -30
    elif view == "right":
        yaw, pitch = -90, -30
    elif view == "rear":
        yaw, pitch = 180, -30
    else:
        if yaw is None and pitch is None:
            print('If view is none, you should pass yaw and pitch. otherwise, shows default values!')
            yaw, pitch = 0, -89

    self.view_matrix = p.computeViewMatrixFromYawPitchRoll(
        cameraTargetPosition=self.target, distance=self.distance, yaw=yaw, pitch=pitch, roll=roll, upAxisIndex=2
    )
    return self.view_matrix
```

For Capturing the image itself i am using following code. It calls Pybullet's getCameraImage() api to capture the image using given view and projection matrix.

```python
def capture(self, filename=None):
    """Capture current view and save image"""
    width, height, rgbImg, depthImg, segImg = p.getCameraImage(
        width=640, height=480, viewMatrix=self.view_matrix, projectionMatrix=self.proj_matrix
    )
    rgbImg = np.reshape(rgbImg, (height, width, 4)).astype(np.uint8)
    img = rgbImg[:, :, :3]

    if filename is None:
        filename = "out/capture.png"
    os.makedirs(os.path.dirname(filename), exist_ok=True)
    plt.imsave(filename, img)
    print(f"Saved: {filename}")
```

Both of these functions are hold together by my camera class which is defined as below:

```python
class Camera:
    def __init__(self, target=[0,0,0], distance=5, fov=60):
        self.target = target
        self.distance = distance
        self.view_matrix = None
        self.proj_matrix = p.computeProjectionMatrixFOV(
            fov=fov, aspect=1.0, nearVal=0.1, farVal=20
        )

    def set_distance(self, distance: int):
        self.distance = distance
```

2.4 Two wheel robot

Finally my robot class which holds both my scene and camera class together, which makes it like main class. I can create instance of the robot class, load scene and robot in it, use camera class to capture images from different angles, i can also make the robot move its position using Pybullet's resetBasePositionAndOrientation() function.

```
class Robot:
    def __init__(self, urdf_path="two_wheel_robot.urdf"):
        p.connect(p.DIRECT)
        p.setGravity(0,0,-9.8)
        self.start_x, self.start_y, self.start_z = -1, -1, 2
        # load scene
        self.scene = Scene(start_x, start_y)
        self.scene.build_platforms()
        # load robot
        self.robot_id = p.loadURDF(urdf_path, [self.start_x, self.start_y, self.start_z])
        # attach camera
        self.camera = Camera(target=[0,0,0], distance=10)
    def move_to(self, pos, quaternion=[0, 0, 0, 1]):
        """Teleport robot to given pos"""
        p.resetBasePositionAndOrientation(self.robot_id, pos, quaternion)
    def teleport(self, bounds=[-2,2]):
        """Teleport robot to random location inside given bounds"""
        x = random.uniform(bounds[0], bounds[1])
        y = random.uniform(bounds[0], bounds[1])
        self.move_to([x, y, self.start_z])
    def disconnect(self):
        p.disconnect()
```

CHAPTER THREE - EXECUTION AND TESTING

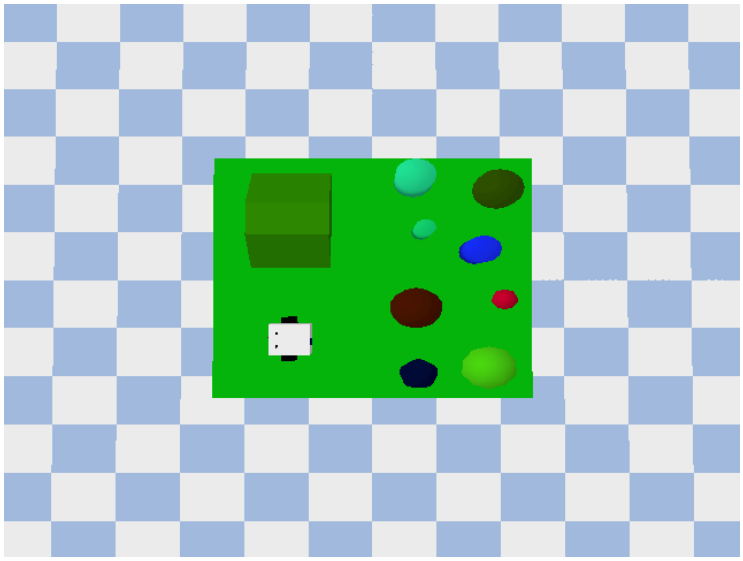
My main script creates the robot, which builds the scene. Then I moved the robot to three different places. At each place, I took a picture from a different view ("rear", "left", "top"). This way, the robot is in a new position in each picture, but the world is static, as the assignment asked

```python
import math
sim = Robot()
# Move robot to 3 positions, capture from top/left/right
positions = [[-1,-1, 2], [1, 1, 2], [1,-1, 2], [-1, 1, 2.5]]
views = ["top", "left", "rear", "right"]

for i, pos in enumerate(positions):
    if i == 3:
        yaw_angle_rad_3 = -90 * (math.pi / 180) # Negative angle for right turn
        orientation3 = p.getQuaternionFromEuler([0, 0, yaw_angle_rad_3])
        sim.move_to(pos, orientation3)
        sim.camera.set_view(view=views[2])
        sim.camera.capture(f"out/scene_{i+1}.png")
    else:
        sim.move_to(pos)
        sim.camera.set_view(view=views[i])
        sim.camera.capture(f"out/scene_{i+1}.png")

from PIL import Image
from IPython.display import display

outf = 'out'
imgs = [os.path.join(outf, i) for i in os.listdir(outf) if i.endswith(('jpg', 'png'))]

for i in imgs:
    display(Image.open(i))

sim.disconnect()
```

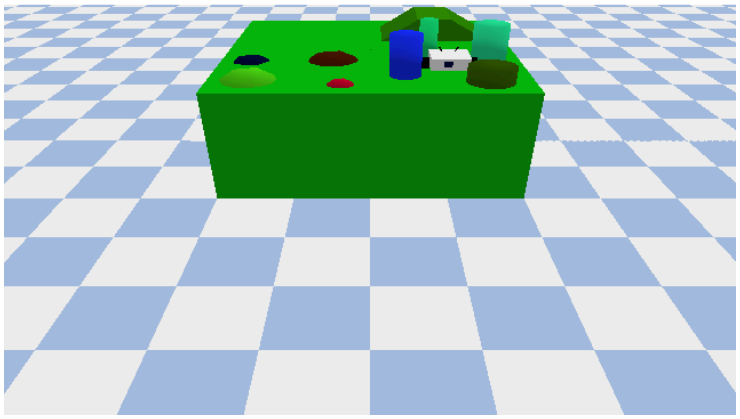
```
Saved: out/scene_1.png
Saved: out/scene_2.png
Saved: out/scene_3.png
Saved: out/scene_4.png
```

3. 1 Screenshots:

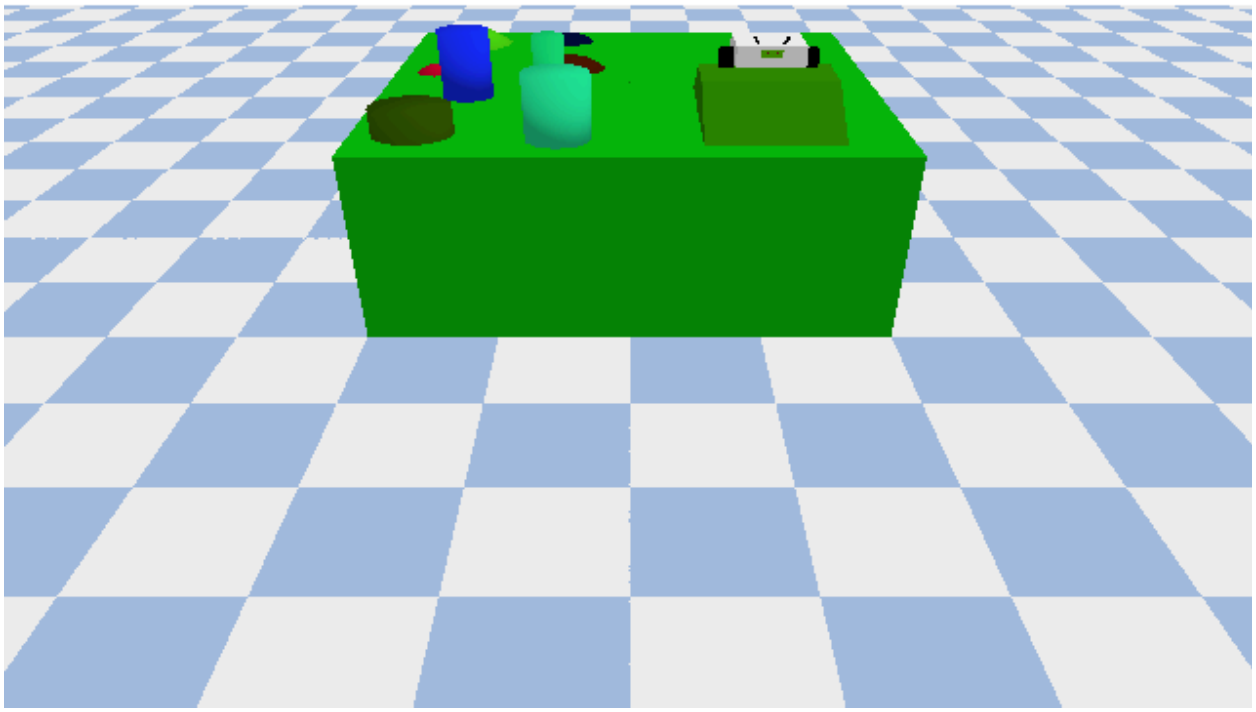
Top View:



Side view:



Another side view:



3.2 Testing

Test Case ID	Description	Procedure	Expected Outcome	Actual Outcome	Status
TC-T1-01	URDF Model Loading	Run the script that loads all custom URDFs (platform_1 to	All models load into the PyBullet simulation without any errors.	All models loaded successfully.	Pass
TC-T1-02	Scene Composition	After loading, check the positions of the four platforms and the	The platforms are arranged in a 2x2 grid, and the robot is placed	The scene was built exactly as planned in the	Pass
TC-T1-03	Camera View 1 (Rear)	Capture the first image with the "rear" view setting.	The image shows the scene from behind the robot. All platforms	The image was captured correctly, showing the rear	Pass
TC-T1-04	Camera View 2 (Left)	Move the robot to the second position and capture the image	The robot is in a new position. The image shows the scene from	The robot moved, and the left-side view was	Pass
TC-T1-05	Camera View 3 (Top)	Move the robot to the third position and capture the image	The robot is in its final position. The image shows a top-down	The robot moved again, and the top-down view	Pass

Table 2: Test cases

CHAPTER FOUR - EVALUATION AND REFLECTION

4.1 Challenges Faced and Solutions

4.1.1 Challenge 1: URDF Complexity

At first, I tried to make simple URDFs with only one shape. But for my final scene , I created platforms with a few more obstacles on them. Making a URDF with many `<link>` and `<joint>` tags was confusing. Getting the positions and orientations right using the `<origin>` tag was difficult and took a lot of trial and error.

Solution: I solved this by building the URDFs piece by piece. I add one obstacle, check it in a visualizer, and then add the next one. This iterative process helped me fix mistakes one by one instead of having many errors at the end.

4.1.2 Challenge 2: Camera Angles

The assignment required capturing the scene from three different angles where all objects are visible. Finding the right values for

`yaw`, `pitch`, `roll`, and `distance` in the `computeViewMatrixFromYawPitchRoll` function was not easy. My first few attempts had some objects outside the picture or hidden behind others.

Solution: I experimented with different different camera parameters. Changing values of `pitch` or `distance`) and then capturing the image. This taught how each parameter changes the view.

4.1.3 Challenge 3: Overwriting Pybullet objects

While doing the assignment, one small issue occurred to me which was a little difficult to debug. So issue was like this, if I load an URDF with specific visuals and then if i try to overwrite that with new visuals, it never used to update the URDF.

Solution: I only realized later that I need to do p.disconnect() in order to fully reset all the URDFs which were previously loaded.

4.2 Potential Improvements

4.2.1 More Dynamic Obstacles:

Right now, all the platforms and obstacles are static because I used `fixed` joints. A big improvement would be to use other joint types like `revolute` (for spinning obstacles) or `prismatic` (for sliding obstacles). This would make the scene more like a real game and a better testing ground for a robot.

CONCLUDING REMARKS

Here, I created a custom scene in PyBullet with my own URDF models, which was like a game of "Takeshi's Castle". I learned how to make complex models and take pictures from different camera views. I learned to create URDF files from scratch, and set up a PyBullet environment, and navigate our robot in it.

Task 2: Autonomous Robot Simulation with ROS2 (TheConstruct.ai)

CHAPTER ONE - PROJECT DEFINITION

1.1 Project Structure

We had to create an autonomous robot system that can:

- Navigate autonomously in a simple environment
- Detect and avoid obstacles in real-time
- Use different sensors such as Camera, LiDAR, IMU. From which we have selected camera and LiDAR for this implementation.
- Build an FSM (≥5 states) controlling robot behavior. We used 5 states.
- Provide human-robot interaction for the manual input.

1.2 Tools and Libraries Used

- **ROS2 Humble:** Core robotics framework
- **Python 3.8+**: Primary programming language
- **OpenCV**: Computer vision for object detection
- **NumPy 1.26.4**: Mathematical computations (downgraded for compatibility)
- **Gazebo**: Physics-based simulation environment
- **TurtleBot3**: Standard mobile robot platform

This report summarizes the process of designing, implementing, executing and evaluating such systems in detailed manner.

CHAPTER TWO - DESIGN AND IMPLEMENTATION

2.1 Environment and robot/object descriptions

Simulation: Gazebo (via gazebo_ros)

Robot: TurtleBot3 Waffle Pi (default model), with following sensors:

- 360° LIDAR (/scan, sensor_msgs/LaserScan)
- Front RGB camera (/camera/image_raw, sensor_msgs/Image)

World: Custom SDF .world file my_robo_world.world. Maze is enclosed and contains inner walls and corridors. A blue cube (0.5m side) is placed at the goal location inside the maze.

Files (important):

- worlds/my_robo_world.world (custom maze)
- launch/my_robo_world.launch.py (launches Gazebo with our world)
- autonomous_bot/behavior_controller_node.py (FSM node)
- run.sh (helper script to build and run)

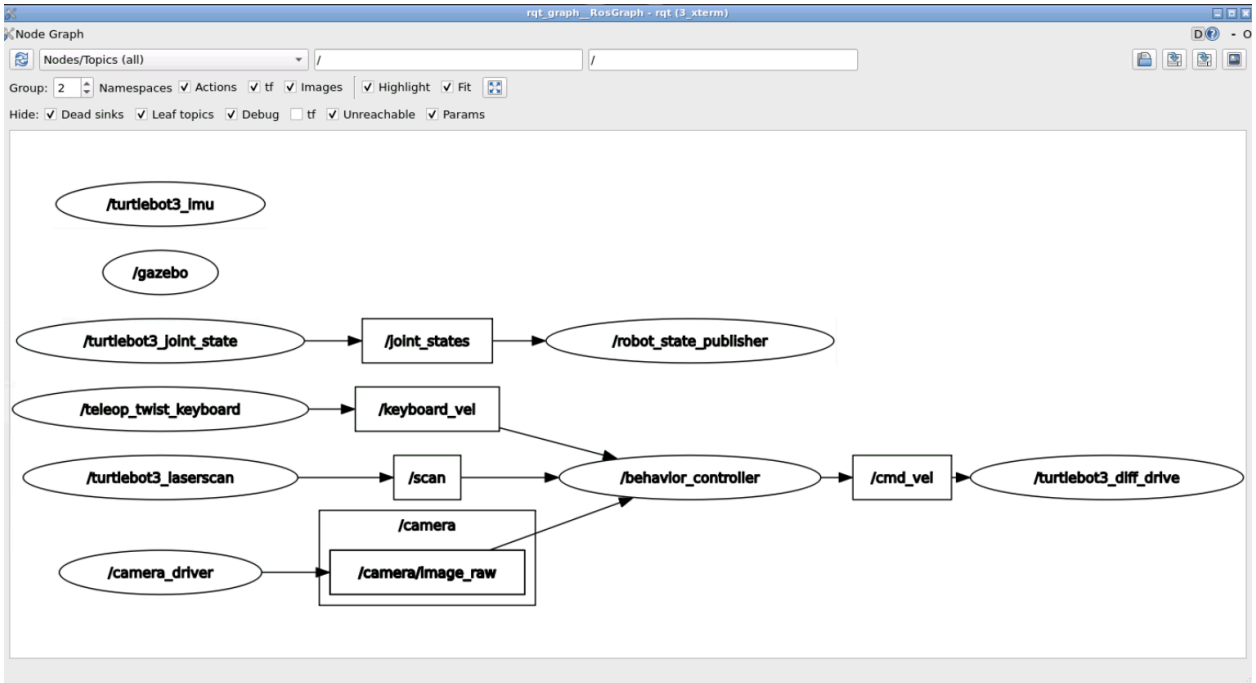


Fig 1: rqt_graph showing all the node and topics

2.2 System Components

ROS2 Nodes:

- `/gazebo`: Simulation environment with physics and sensors
- `/robot_state_publisher`: TF transformations for robot kinematics
- `/behavior_controller`: Custom FSM controller (our implementation)
- `/teleop_twist_keyboard`: Manual control interface

ROS2 Topics:

- `/scan`: LIDAR sensor data (sensor_msgs/LaserScan)
- `/camera/image_raw`: Camera feed (sensor_msgs/Image)
- `/cmd_vel`: Final velocity commands (geometry_msgs/Twist)
- `/keyboard_vel`: Manual control commands (geometry_msgs/Twist)

2.3 Sensor Integration:

2.3.1 LIDAR Integration

Sensor: LDS-01 360-degree LIDAR (built into TurtleBot3 Waffle Pi) Topic: /scan (sensor_msgs/msg/LaserScan) Integration:

```
self.scan_subscriber = self.create_subscription(
    LaserScan, '/scan', self.scan_callback,
qos_profile_sensor_data)
```

Processing:

- Extracts minimum distance in forward 60-degree arc
- Filters out invalid readings (infinity/NaN values)
- Provides real-time obstacle detection

2.3.2 Camera Integration

Sensor: RGB Camera (built into TurtleBot3 Waffle Pi)

Topic: /camera/image_raw (sensor_msgs/msg/Image)

Integration:

```
self.camera_subscriber = self.create_subscription(
    Image, '/camera/image_raw', self.camera_callback,
10)
```

Processing:

- Converts ROS image to OpenCV format using CvBridge
- Applies HSV color filtering for blue object detection
- Uses contour detection to locate and track goals
- Determines goal position (left/right/center) relative to robot

2.3.3 Human-Robot Interaction (HRI)

Keyboard Teleoperation

Package: teleop_twist_keyboard Command:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
--ros-args --remap cmd_vel:=keyboard_vel
```

2.3.4 HRI Features:

Direct Control: keys for movement (forward, backward, turn) uses /keyboard_vel topic

Speed Control: Q/Z to increase/decrease speed

2.4 Key algorithms & how applied

2.4.1 LIDAR obstacle detection

- From `LaserScan.ranges`, we examine a forward arc (±30°) to compute `min_lidar_dist`.
- If `min_lidar_dist < threshold` (0.4 m), then OBSTACLE_AVOIDANCE is triggered (high priority).

2.4.2 Camera-based goal detection (color segmentation)

- Convert camera frame to HSV
- Use threshold: `BLUE_LOWER = [100,150,50], BLUE_UPPER = [140,255,255]` to get mask.
- Find contours; select largest contour (if area > 500 px).
- Compute bounding rectangle: `x, y, w, h`; center x `cx`.
- Decide `GOAL_LEFT` / `GOAL_RIGHT` / `GOAL_CENTER` based on `cx` vs image center ± dead zone.
- We also compute normalized size `(w*h)/(image_area)` to estimate proximity.

2.4.3 Finite State Machine (FSM)

Primary state:

- `IDLE` — robot stopped
- `MANUAL_CONTROL` — forward keyboard commands
- `AUTONOMOUS_SEEKING` — camera-based goal seeking
- `OBSTACLE_AVOIDANCE` — reactive turning to avoid collision
- `GOAL_FOUND` — stop after reaching goal

Current	Trigger	Next	Action
Any	min_lidar_dist < 0.4 and goal not centered	OBSTACLE_AVOIDANCE	stop linear; rotate away
IDLE	keyboard input (recent)	MANUAL_CONTROL	forward keyboard commands
IDLE	goal visible (camera)	AUTONOMOUS_SEEKING	start seeking behaviour
MANUAL_CONTROL	no keyboard input >2s	IDLE	publish zero velocity
AUTONOMOUS_SEEKING	center & close	GOAL_FOUND	stop robot
AUTONOMOUS_SEEKING	goal lost	IDLE	stop
OBSTACLE_AVOIDANCE	obstacle cleared	AUTONOMOUS_SEEKING	resume seeking
GOAL_FOUND	manual reset	IDLE	wait for next run

Table 1. State Transition table (behavior_controller_node.py)

CHAPTER THREE - EXECUTION AND TESTING

3.1 How to run (TheConstruct.ai RosJect)

1. Open terminal in rosject.

2. Set TurtleBot3 model, build workspace:
   source /opt/ros/humble/setup.bash

   cd ~/ros2_ws

   colcon build

   source install/setup.bash

   export TURTLEBOT3_MODEL=waffle_pi

3. Launch Gazebo with our world:
   ros2 launch autonomous_bot my_robo_world.launch.py
4. Start behavior controller (in another terminal):

   ros2 run autonomous_bot behavior_controller_node

5. Start manual teleop (foreground):
   ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args --remap cmd_vel:=keyboard_vel

We have also created a single bash script `./run.sh` which builds and runs all steps.

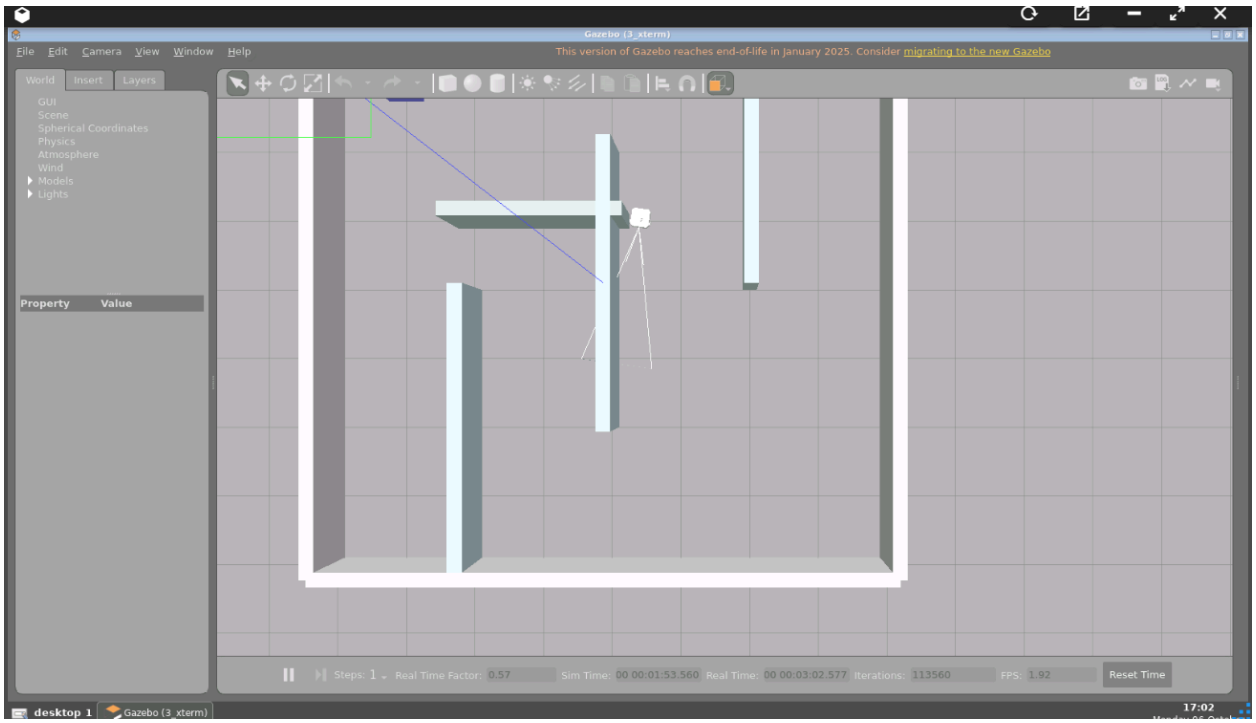
3.2 Test cases & outcomes

We have created and ran the following small set of test cases.

Test Case ID	Description	Procedure	Expected Outcome	Actual Outcome	Status
TC-01	Manual Control Verification	Launch system and teleop node. Press keys for forward, left turn, and right turn.	Robot responds correctly to all keyboard commands.	Robot moves forward, turns left, and turns right as commanded.	Pass
TC-02	Head-on Obstacle Avoidance	Place robot 2m from a wall. Start autonomous seeking.	Robot moves towards the wall, stops at >0.4m, turns away, and continues.	Robot stopped at 0.45m, turned 90 degrees left, and resumed seeking.	Pass
TC-03	Goal Acquisition	Place a green object 3m in front of the robot. Start autonomous seeking.	Robot orients towards the object, drives to within 0.2m, and enters GOAL_FOUND state.	Robot successfully navigated to the object and stopped.	Pass
TC-04	State Transition Priority	While the robot is seeking a goal, manually place a box in its path.	Robot transitions from AUTONOMOUS_SEEKING to OBSTACLE_AVOIDANCE, maneuvers around the box, then transitions back to AUTONOMOUS_SEEKING.	System correctly prioritized obstacle avoidance over goal seeking.	Pass

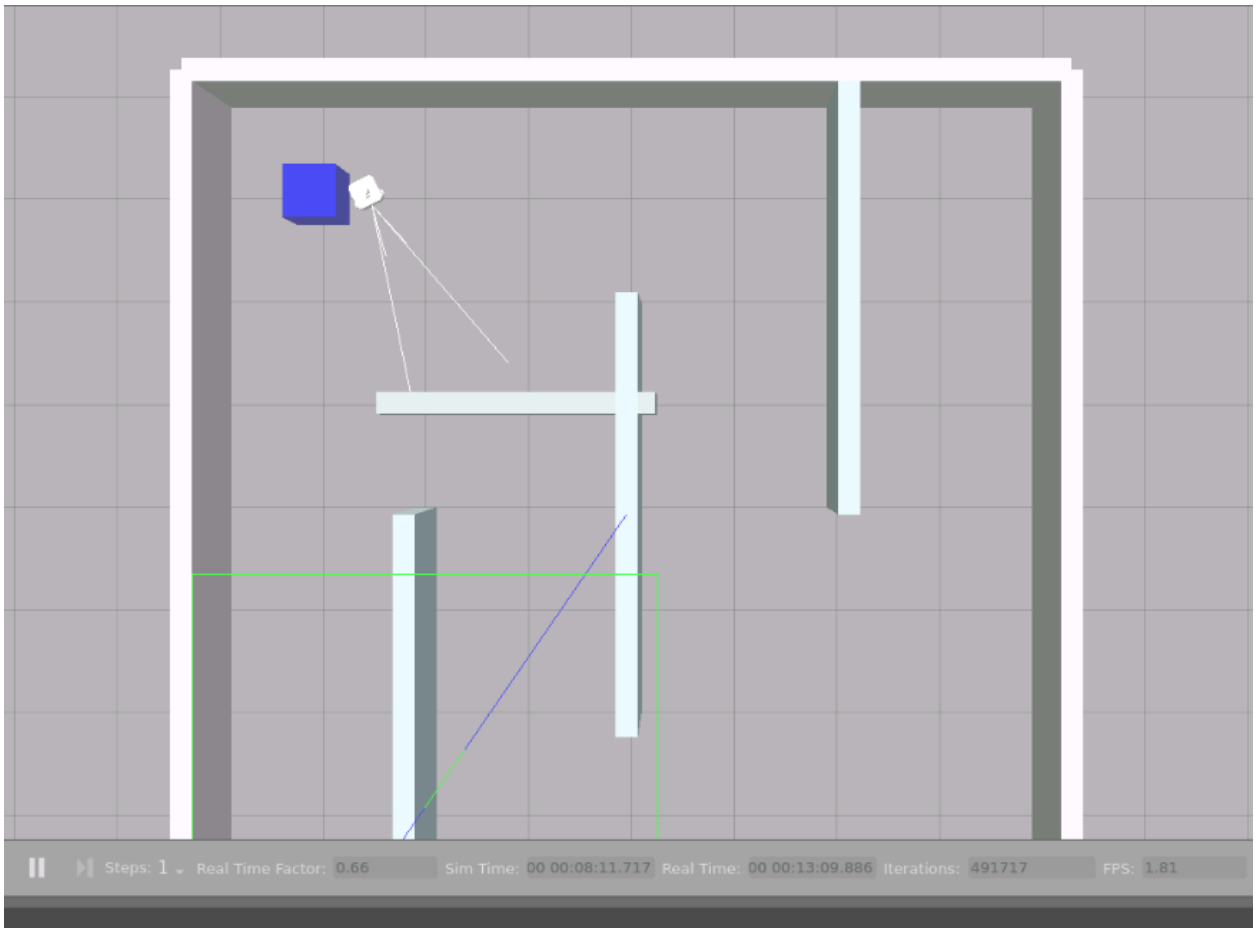
Table 2: Sample Test Cases

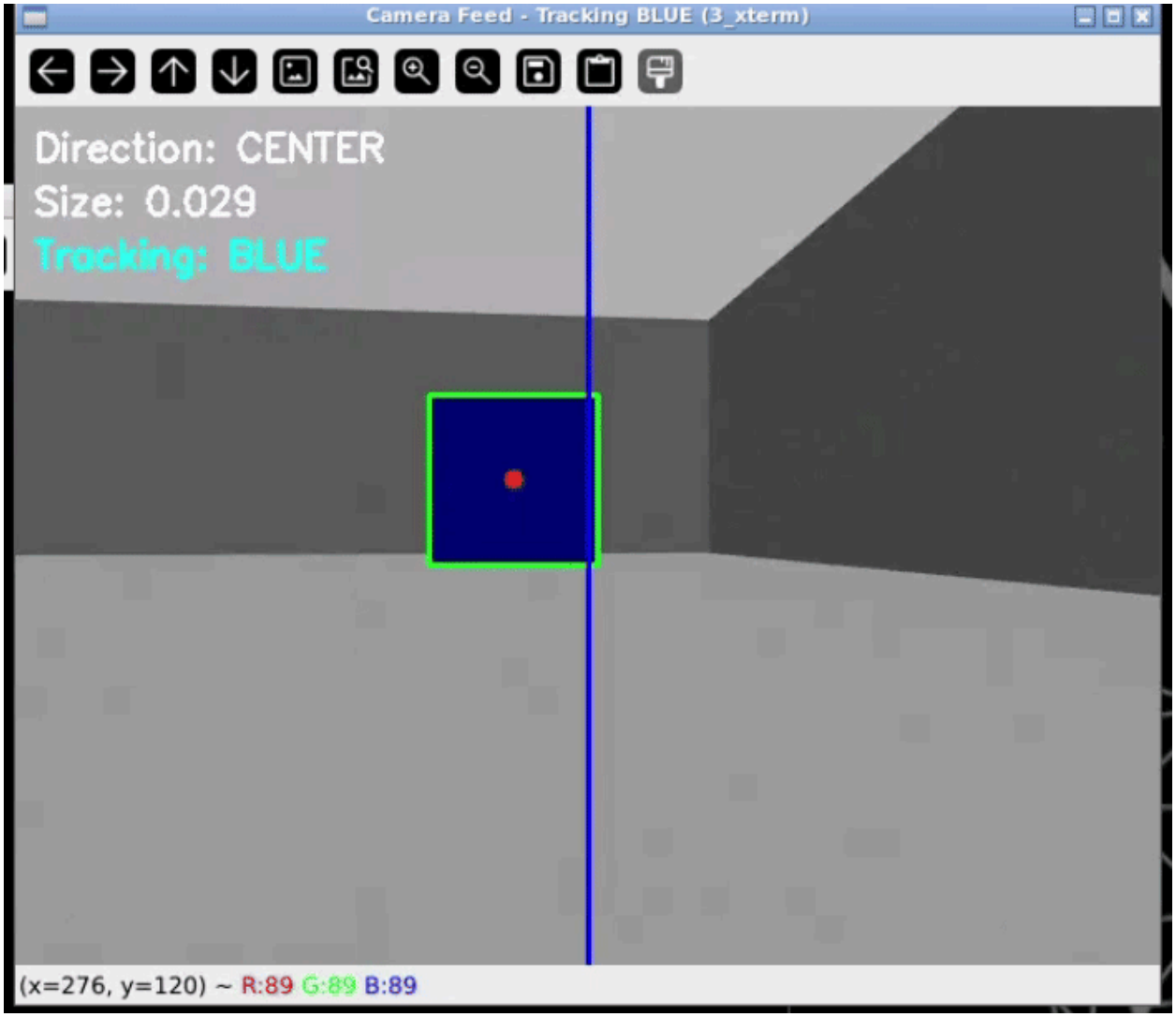
3.3 Screenshots



State Transitions:

```
[INFO] [1759770771.036791537] [behavior_controller]:
State transition: OBSTACLE_AVOIDANCE -> AUTONOMOUS_SEEKING
[INFO] [17
59770771.104066446] [behavior_controller]: State transition: AUTONOM
OUS_SEEKING -> IDLE
[INFO] [1759770771.117560010] [behavior_controlle
r]: State transition: AUTONOMOUS_SEEKING -> IDLE
[INFO] [1759770771.1
89020131] [behavior_controller]: State transition: AUTONOMOUS_SEEKIN
G -> IDLE
[INFO] [1759770842.004546179] [behavior_controller]: State transition: IDLE -> MANUAL_CONTROL
[INFO] [1759770842.
015416196] [behavior_controller]: State transition: IDLE -> MANUAL_CONTROL
[INFO] [1759770842.088614570] [behavior_control
ler]: State transition: IDLE -> MANUAL_CONTROL
[INFO] [1759770844.004832505] [behavior_controller]: State transition: MANU
AL_CONTROL -> IDLE
[INFO] [1759770844.015298550] [behavior_controller]: State transition: MANUAL_CONTROL -> IDLE
[INFO] [17
59770844.089763615] [behavior_controller]: State transition: MANUAL_CONTROL -> IDLE
```


Steps: 1
Real Time Factor: 0.66
Sim Time: 00 00:08:11.717
Real Time: 00 00:13:09.886
Iterations: 491717
FPS: 1.81


Camera Feed - Tracking BLUE (3_xterm)
Direction: CENTER
Size: 0.029
Tracking: BLUE
(x=276, y=120) ~ R:89 G:89 B:89

CHAPTER FOUR - EVALUATION AND REFLECTION

4.1 Challenges faced and solutions

4.1.1 URDF wall orientation & texture

- Problem: walls overlapped / not forming enclosure (rotation missing).
- Solution: rotate two walls by 90° so long axis aligns correctly; tile multiple 1×1 wall segments to avoid stretched texture.

4.1.2 ROS2 packaging & launch

- Problem: `ros2 launch` could not find custom launch/world in installed package.
- Solution: update `setup.py` to include `data_files` for `launch/*.launch.py` and `worlds/*.world`. Rebuild with `colcon build` and `source install/setup.bash`.

4.1.3 Wrong Gazebo launch file

- Problem: using `turtlebot3_world.launch.py` ignored `world` argument.
- Solution: use `gazebo_ros/launch/gazebo.launch.py` and pass `world` path.

4.1.4 LIDAR rays clutter

- Problem: blue rays in Gazebo GUI.
- Solution: hide via Gazebo View options during demo.

4.1.5 Camera/goal misclassification

- Problem: LIDAR triggered avoidance when approaching goal cube.
- Solution: in FSM, if goal is centered, suppress immediate avoidance; additional threshold tuning.

4.2 Potential improvements

- **Use depth camera or stereo** to estimate distance more robustly instead of area heuristic.
- **SLAM and mapping**: build map (gmapping / cartographer) and plan path rather than simple reactive behaviors.
- **PID controllers** for smoother approach and turning.

- **Better object detection**: use pretrained neural network for robust detection in varied lighting.
- **Multiple goals and path planning**: A* or Dijkstra for more complex navigation.
- **Tests on real robot**: port code to real TurtleBot3 for real-world validation.

4.3 Reflection: mapping to real-world robotics

This assignment replicates many real-world challenges:

- Sensor noise (LIDAR false positives), need for sensor fusion.
- Software modularity (ROS nodes) matches how industry systems are built.
- Safety-first design (obstacle avoidance highest priority) is critical in real robots.
- HRI and manual override are practical requirements in deployed robots.

CONCLUDING REMARKS

I implemented a ROS2-based autonomous TurtleBot3 that uses LIDAR and camera for safe goal-seeking inside a custom maze. The FSM design prioritized safety and allowed manual override. I documented the setup and resolved multiple practical issues (URDF orientation, texture tiling, ROS2 package installation, Gazebo launch selection, LIDAR vs goal confusion). For future work, I will add SLAM, path planning, and better vision methods.

Challenges:

Numpy Issue:

```
Traceback (most recent call last):  File "/home/user/ros2_ws/install/autonomous_bot/lib/autonom
ous_bot/behavior_controller_node", line 33, in <module>
    sys.exit(load_entry_point('autonomous-bot==0.0.1', 'console_scripts', 'behavior_controller_
node')())
  File "/home/user/ros2_ws/install/autonomous_bot/lib/autonomous_bot/behavior_controller_node",
 line 25, in importlib_load_entry_point
    return next(matches).load()
  File "/usr/lib/python3.10/importlib/metadata/__init__.py", line 171, in load
    module = import_module(match.group('module'))
  File "/usr/lib/python3.10/importlib/__init__.py", line 126, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
  File "/home/user/ros2_ws/install/autonomous_bot/lib/python3.10/site-packages/autonomous_bot/b
ehavior_controller_node.py", line 7, in <module>
    from cv_bridge import CvBridge
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/cv_bridge/__init__.py", line 6, in <
module>
    from cv_bridge.boost.cv_bridge_boost import cvtColorForDisplay, getCvType
AttributeError: _ARRAY_API not found

A module that was compiled using NumPy 1.x cannot be run in
NumPy 2.1.1 as it may crash. To support both 1.x and 2.x
versions of NumPy, modules must be compiled with NumPy 2.0.
Some module may need to rebuild instead e.g. with 'pybind11>=2.12'.

If you are a user of the module, the easiest solution will be to
downgrade to 'numpy<2' or try to upgrade the affected module.
```

BIBLIOGRAPHY

1. Coumans, E. and Bai, Y. (2016-2021). *PyBullet, a Python module for physics simulation for games, robotics and machine learning*. [Online]. [Accessed 6 October 2025]. Available from: http://pybullet.org
2. ROS Index (2024). *teleop_twist_keyboard*. [Online]. [Accessed 6 October 2025]. Available from: https://index.ros.org/p/teleop_twist_keyboard/
3. Github.io. (2025). *Welcome To Zscaler Directory Authentication*. [online] Available at: https://lever-robotics.github.io/URDF_creator/.

APPENDIX (if necessary)

RosJect project unique name: SLIbR8uEC5

Colab link for task 1: Assignment_RSP_PyBullet.ipynb

Github link for the project:
https://github.com/Sanket758/AtBSBI/tree/master/Robotics%20and%20Software%20Programming%20-%20Assignment